

Rapport Projet - Programmation répartie

Alexis CERDA DE ALMEIDA VILACA

Logan SAGET

Taïno HERMANN

Emelyne FOUSSE

4 juin 2026

Table des matières

1	Squelette RMI	2
2	Questions du projet	4
2.1	Que fait le client lors d'un appel à un service?	4

1 Squelette RMI

Annuaire

Commande rmiregistry 1099

```
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
import java.rmi.RemoteException;

public class Annuaire {
    public static void main(String[] args) {
        String adresse = args[0];
        Registry reg;
        try {
            reg = LocateRegistry.getRegistry(adresse);
            for (String elt : reg.list()) {
                System.out.println(elt);
            }
        } catch (RemoteException e) { // lancer par getRegistry
            System.out.println(e.getMessage());
        }
    }
}
```

Service

```
import java.rmi.RemoteException;

public class Service implements ServiceInterface {
    public int Methode() throws RemoteException {
        // le throws est obligatoire pour les methodes d une interface RMI
        String host = ""; // ligne qui permet d'afficher l'IP du client !
        try {
            host = RemoteServer.getClientHost();
            System.out.println(host);
        } catch (ServerNotActiveException e) {} // jusqu'ici
        return 0;
    }
}
```

LancerService

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
import java.rmi.server.ServerNotActiveException;

public class LancerService {
    public static void main(String args[]) throws RemoteException,
        ServerNotActiveException {
        try {
            Service nom = new Service(); // creation d un objet (new)
            ServiceInterface rd = (ServiceInterface) UnicastRemoteObject.
                exportObject(nom, 0); // 0 pour que OS donne le port
                automatiquement
            Registry reg = LocateRegistry.getRegistry(1099); // 1099 port
                par defaut des annuaire (modifiable)
            reg.rebind("LeNom", rd); // donner un nom dans l annuaire pour
                le service
        } catch (RemoteException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

ServiceInterface

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface ServiceInterface extends Remote {
    public int Methode() throws RemoteException; // toutes les
        exceptions ...
}
```

Elle doit être commune entre le Service et le Client

Client

Appel

```
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
import java.rmi.RemoteException;
import java.rmi.NotBoundException;

public class Appel {
    public static void main(String[] args) {
        String adresse = args[0];
        Registry reg;
        try {
            reg = LocateRegistry.getRegistry(adresse);
            ServiceInterface Objet = (ServiceInterface) reg.lookup("LeNom"
);
            int num = Objet.Methode();
        } catch (NotBoundException e) {
            System.out.println("Reference non trouvee dans l'annuaire");
        } catch (RemoteException e2) {
            System.out.println("Reference(s) non cree(s)");
        }
    }
}
```

2 Questions du projet

2.1 Que fait le client lors d'un appel à un service ?

Faire un petit schéma de cette architecture en identifiant les choses suivantes :

Le/les processus fixes (ceux qui écoutent sur un port choisi) et les **processus mobiles** ? (ceux qui rentrent et sortent à leur guise) ?

Les types des données échangées entre les processus

Si on veut que les calculs se fassent en parallèle que faut-il faire ?

Il faut séparer chaque envoi aux noeuds dans des nouveaux Thread.